UML DİYAGRAMLARI

Use Case, ERveClassDiyagramları–Açıklamalarve Örnekler

Bilgi	Açıklama
Adı Soyadı	Berke HARMAN
Öğrenci No	256001044
Örnek Senaryo	Kütüphane Yönetim Sistemi

1. UML Nedir?

UML(UnifiedModeling Language – Birleşik Modelleme Dili), bir yazılım sistemini kod yazmaya başlamadan önce **görsel olarak modellemek, tasarlamak ve belgelemek** için kullanılan standart bir çizim dilidir. 1990'ların ortasında Grady Booch, James Rumbaugh ve Ivar Jacobson'ın kendi yöntemlerini birleştirmesiyle ortaya çıkmış, 1997'de **OMG (Object Management Group)** tarafından standart olarak kabul edilmiştir. Günümüzdeki en güncel sürümü **UML 2.5.1**'dir (2017).

UML bir programlama dili değildir; bir **modelleme dilidir**. Bir binanın inşaatına başlamadan önce mimari proje çizilmesi gibi, yazılımda da UML diyagramları sistemin "mimari projesi" görevini görür. Aynı semboller herkes için aynı anlamı taşıdığından yazılımcı, analist, tasarımcı ve müşteri arasında **ortak bir dil** oluşturur.

1.1 UML Neden Kullanılır?

- **İletişimi kolaylaştırır:** Uzun metinler yerine tek bir diyagramla sistemin yapısı herkes tarafından anlaşılır.
- **Hataları erken yakalar:** Tasarım hataları kod yazılmadan, diyagram aşamasında fark edilir; bu da maliyeti büyük ölçüde azaltır.
- **Belgeleme sağlar:** Projeye sonradan katılan biri diyagramlara bakarak sistemi hızlıca öğrenebilir.
- **Kodlamaya yol gösterir:** Class diyagramları doğrudan sınıflara, ER diyagramları doğrudan veritabanı tablolarına dönüştürülebilir.

1.2 UML Diyagram Türleri

UML 2.5'te toplam 14 diyagram türü bulunur ve bunlar iki ana gruba ayrılır:

Grup	Açıklama	Diyagramlar
Yapısal (Structural) Diyagramlar	Sistemin statik yapısını, yani "nelerden oluştuğunu" gösterir.	Class, Object, Component, Deployment, Package, Composite Structure, Profile
Davranışsal (Behavioral) Diyagramlar	Sistemin dinamik davranışını, yani "nasıl çalıştığını" gösterir.	Use Case, Activity, State Machine, Sequence, Communication, Timing, Interaction Overview

Bu çalışmada en sık kullanılan iki UML diyagramı olan **Use Case** ve **Class** diyagramları ile veritabanı tasarımında kullanılan **ER diyagramı** açıklanmıştır. ER diyagramı aslında UML'in resmi bir parçası değildir (Peter Chen, 1976), ancak yazılım projelerinde UML diyagramlarıyla birlikte kullanıldığı için ele alınmıştır. Üç diyagramın da aynı senaryo üzerinden karşılaştırılabilmesi için örnek olarak bir **Kütüphane Yönetim Sistemi** seçilmiştir.

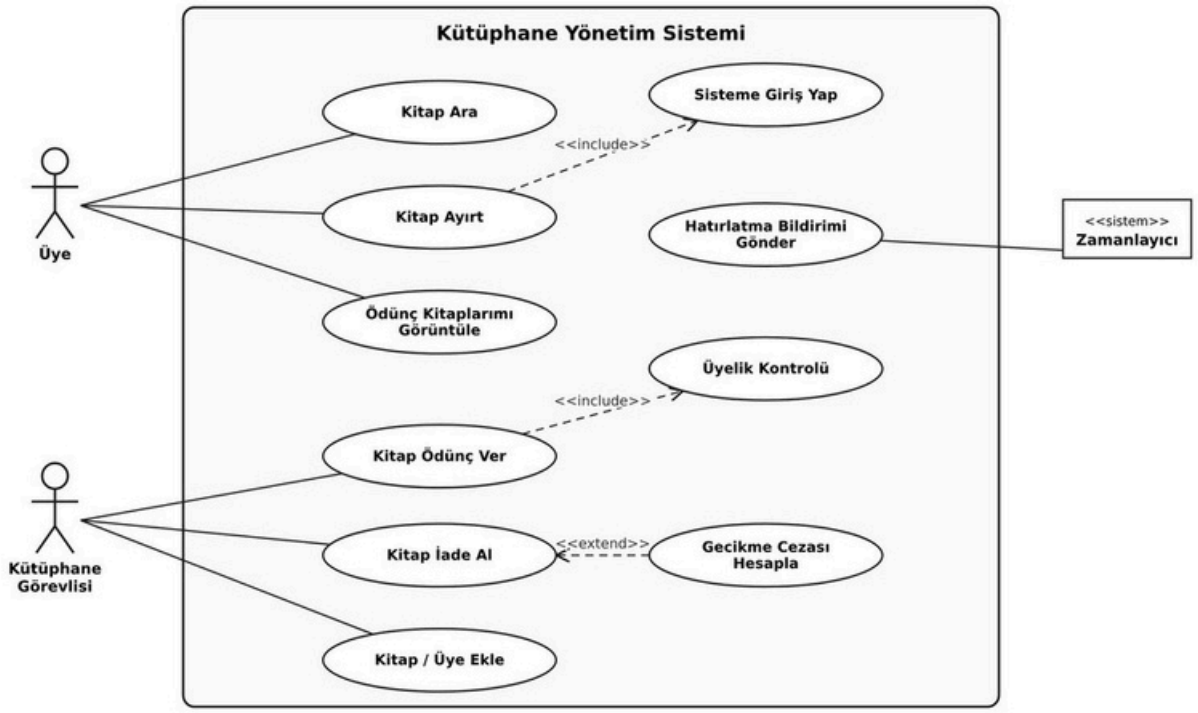
2. Use Case (Kullanım Durumu) Diyagramı

Use Case diyagramı, sistemi **kullanıcının bakışaçısından**gösteren davranışsal bir diyagramdır. "Sistemi kimler kullanacak ve sistemle neler yapabilecekler?" sorusunun cevabını verir. Sistemin içinin nasıl çalıştığıyla ilgilenmez; sadece **ne yaptığını** gösterir. Bu nedenle genellikle gereksinim analizi aşamasında, müşteriyle birlikte çizilir.

2.1 Temel Elemanlar

Eleman	Gösterimi	Açıklama
Aktör (Actor)	Çöp adam	Sistemle etkileşime giren kişi, rol veya dış sistem. Aktör bir kişi değil, bir roldür (Üye, Görevli, Zamanlayıcı).
Kullanım Durumu (Use Case)	Elips (oval)	Aktörün sistemde gerçekleştirdiği, ona değer sağlayan bir işlev. Fiil ile adlandırılır (Kitap Ara, Kitap Ödünç Ver).
Sistem Sınırı (System Boundary)	Dikdörtgen	Sistemin kapsamını gösterir. Use case'ler içeride, aktörler dışarıda yer alır.
İlişki (Association)	Düz çizgi	Aktörün hangi use case'i kullandığını gösterir.
<<include>>	Kesikli ok	Bir use case'in çalışması için başka bir use case'in her zaman çalışması gerektiğini gösterir. Ok, dahil edilen use case'e doğrudur.
<<extend>>	Kesikli ok	Bir use case'in sadece belirli bir koşulda başka bir use case'e ek davranış eklediğini gösterir. Ok, genişletilen (ana) use case'e doğrudur.
Genelleme (Generalization)	İçi boş üçgen uçlu ok	Bir aktörün veya use case'in başka birinin özel hâli olduğunu gösterir.

2.2 Örnek: Kütüphane Yönetim Sistemi Use Case Diyagramı



Şekil 1. Kütüphane Yönetim Sistemi – Use Case Diyagramı

2.3 Diyagramın Açıklaması

- **Aktörler:** Sistemde üç aktör vardır. **Üye**, kitap arar, ayırtır ve ödünç aldığı kitapları görüntüler. **Kütüphane Görevlisi**, kitap ödünç verir, iade alır ve sisteme yeni kitap veya üye ekler. **Zamanlayıcı**

ise bir insan değil, belirli zamanlarda otomatik çalışan bir **sistem aktörüdür**; teslim tarihi yaklaşan üyelere hatırlatma bildirimi gönderir.

<<include>> örneği: "Kitap Ayırt" işlemi yapılabilmesi için üyenin **her seferinde** sisteme giriş yapmış olması gerekir. Benzer şekilde "Kitap Ödünç Ver" işleminde **her zaman** üyelik kontrolü yapılır (üyelik aktif mi, en fazla 3 kitap sınırı aşılmış mı?).

<<extend>> örneği: "Gecikme Cezası Hesapla", kitap iade alınırken **sadece kitap geç getirildiyse** çalışır. Kitap zamanında getirilirse bu adım hiç gerçekleşmez. Bu nedenle bir extend ilişkisidir.

- **Sistem sınırı:** Dikdörtgenin içindeki işlevler kütüphane yazılımının kapsamındadır; aktörler sistemin dışında yer alır.

İpucu: include ile extend'i ayırt etmek için şu soru sorulabilir: "Bu adım **her zaman** mı çalışır, yoksa **bazen** mi?" Her zaman çalışıyorsa include, bir koşula bağlıysa extend kullanılır.

3. ER (Entity-Relationship / Varlık-İlişki) Diyagramı

ER diyagramı, bir sistemde saklanacak **verileri** ve bu veriler arasındaki **ilişkileri** gösteren bir veritabanı tasarım diyagramıdır. 1976 yılında **Peter Chen** tarafından önerilmiştir. Bir ER diyagramı neredeyse birebir olarak ilişkisel veritabanı tablolarına dönüştürülebilir. Bu sayede tablolar oluşturulmadan önce hangi tabloların, hangi alanların ve hangi ilişkilerin olacağı netleştirilir; tekrar eden veri ve tutarsızlık problemleri tasarım aşamasında çözülür.

3.1 Temel Kavramlar

Kavram	Açıklama
Varlık (Entity)	Hakkında bilgi saklanan nesne veya kavram. Veritabanında tabloya karşılık gelir (UYE, KITAP, ODUNC).
Nitelik (Attribute)	Varlığın özellikleri. Tablodaki sütunlara karşılık gelir (Baslik, ISBN, BasimYili).
Birincil Anahtar (PK)	Her kaydı benzersiz şekilde tanımlayan alan (KitapId). Boş ve tekrarlı olamaz.
Yabancı Anahtar (FK)	Başka bir tablonun birincil anahtarına başvuran alan. Tablolar arasındaki ilişkiyi kurar (ODUNC.UyeId → UYE.UyeId).
Benzersiz Anahtar (UK)	Birincil anahtar olmasa da tekrarlanamayan alan (ISBN, Eposta).
İlişki (Relationship)	İki varlık arasındaki bağlantı (Üye kitap alır, Kategori kitap içerir).
Kardinalite	Bir varlıktaki kaydın diğer varlıktaki kaç kayıtla ilişkili olabileceği: 1-1, 1-N, N-M.

3.2 İlişki Türleri (Kardinalite)

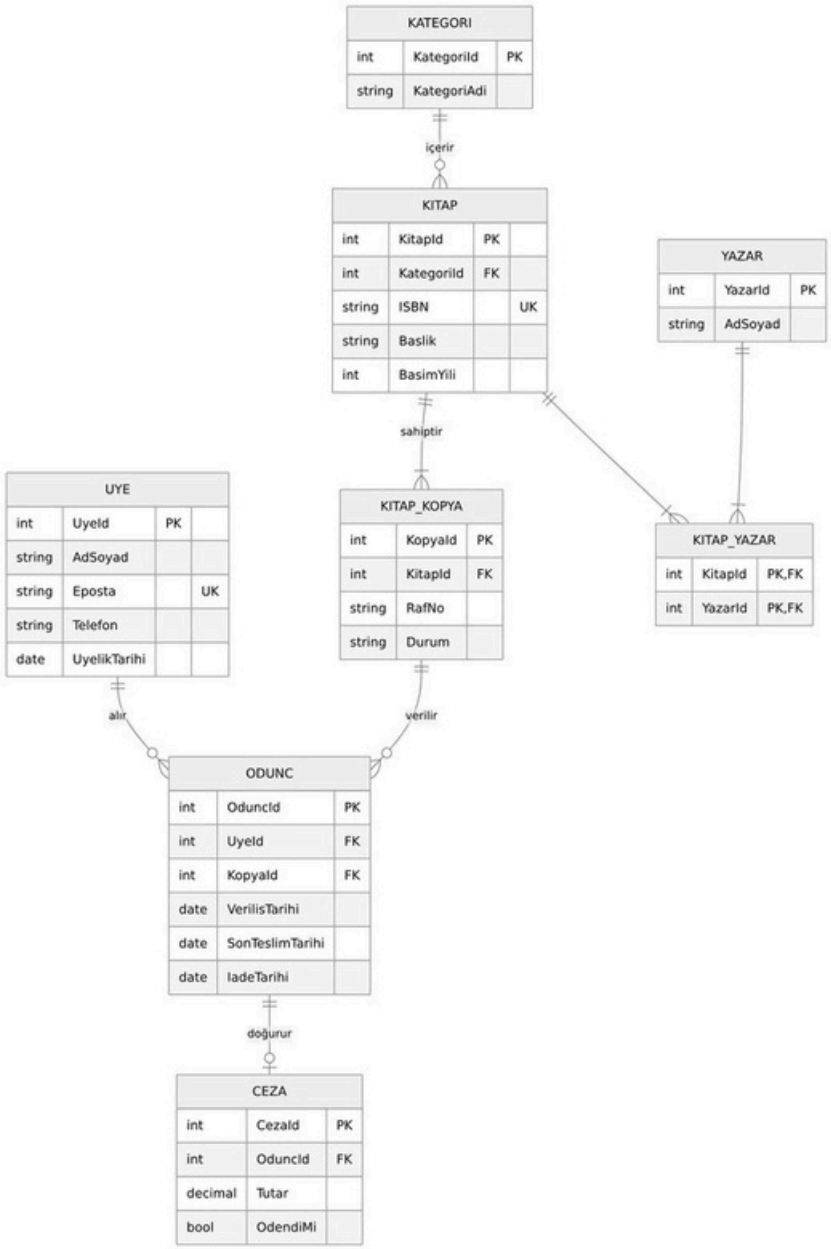
- **Bire-bir (1:1):** Bir kayıt, diğer tablodaki en fazla bir kayıtla ilişkilidir. Örnek: Bir ödünç işleminin en fazla bir cezası olabilir, bir ceza da sadece bir ödünç işlemine aittir.
- **Bire-çok (1:N):** Bir kayıt, diğer tablodaki birçok kayıtla ilişkilidir. Örnek: Bir kategoride birçok kitap bulunur, ama her kitap tek bir kategoriye aittir. Bu ilişkide yabancı anahtar "çok" tarafına (KITAP tablosuna) konur.
- **Çoka-çok (N:M):** Her iki taraftaki kayıtlar birden fazla kayıtla ilişkili olabilir. Örnek: Bir kitabın birden fazla yazarı olabilir, bir yazar da birden fazla kitap yazmış olabilir. İlişkisel veritabanında N:M ilişki doğrudan kurulamaz; araya bir **ara tablo (KITAP_YAZAR)** eklenerek iki adet 1:N ilişkiye dönüştürülür.

3.3 Crow's Foot (Karga Ayağı) Gösterimi

Bu çalışmadaki diyagramda günümüzde en yaygın kullanılan **Crow's Foot** gösterimi kullanılmıştır. Çizginin her iki ucundaki semboller, o taraftaki en az ve en çok kayıt sayısını gösterir:

Sembol	Anlamı
|| (iki dik çizgi)	Tam olarak bir (zorunlu bir)
o| (daire + çizgi)	Sıfır veya bir (isteğe bağlı)
|< (çizgi + karga ayağı)	Bir veya daha fazla (en az bir zorunlu)
o< (daire + karga ayağı)	Sıfır veya daha fazla

3.4 Örnek: Kütüphane Yönetim Sistemi ER Diyagramı



Şekil2.Kütüphane Yönetim Sistemi – ER Diyagramı (Crow's Foot gösterimi)

3.5 Diyagramın Açıklaması

Diyagramda8 varlık(tablo)bulunmaktadır. İlişkiler şöyle okunur:

İlişki	Tür	Açıklama
KATEGORI – KITAP	1:N	Bir kategoride sıfır veya daha fazla kitap olabilir; her kitap tam olarak bir kategoriye aittir.

İlişki	Tür	Açıklama
KITAP – KITAP_KOPYA	1:N	Kütüphanede aynı kitaptan birden fazla fiziksel kopya olabilir. Kitap bilgisi (başlık, ISBN) bir kez saklanır, kopyalar ayrı tutulur.
KITAP – YAZAR	N:M	KITAP_YAZAR ara tablosu ile çözülmüştür. Ara tablonun birincil anahtarı iki yabancı anahtarın birleşimidir (bileşik anahtar).
UYE – ODUNC	1:N	Bir üye zaman içinde birçok ödünç işlemi yapabilir; her ödünç işlemi tek bir üyeye aittir.
KITAP_KOPYA – ODUNC	1:N	Bir kopya farklı zamanlarda birçok kez ödünç verilebilir.
ODUNC – CEZA	1:1 (isteğe bağlı)	Kitap geç iade edilirse ödünç kaydına bir ceza bağlanır, zamanında iade edildiyse ceza oluşmaz.

Neden KITAP ve KITAP_KOPYA ayrı? Eğer her kopya için KITAP tablosuna ayrı satır eklenseydi aynı başlık, ISBN ve yazar bilgisi defalarca tekrarlanacaktı. Bu tasarım **normalizasyon** kurallarına uygundur ve veri tekrarını önler.

4. Class (Sınıf) Diyagramı

Class diyagramı, nesne yönelimli bir sistemdeki **sınıfları**, bu sınıfların **özelliklerini (attribute)** ve **metotlarını (operation)**, ayrıca sınıflar arasındaki **ilişkileri** gösteren yapısal bir UML diyagramıdır. UML'de en çok kullanılan diyagramdır ve doğrudan koda (C#, Java vb.) dönüştürülebilir.

ER diyagramı ile benzer görünse de amacı farklıdır: ER diyagramı **verinin nasıl saklanacağını**, class diyagramı ise **yazılımın nasıl çalışacağını** modeller. Bu yüzden class diyagramında veritabanında olmayan metotlar, arayüzler (interface), soyut sınıflar ve servis sınıfları da yer alır.

4.1 Sınıf Gösterimi

Bir sınıf üç bölmeli bir dikdörtgenle gösterilir: en üstte **sınıf adı**, ortada **özellikler**, en altta **metotlar**. Özellikler `görünürlük ad: tip`, metotlar `görünürlük ad(parametreler): dönüş tipi` şeklinde yazılır. Soyut sınıflar ve metotlar *italik* veya «abstract», arayüzler «interface» ile belirtilir.

Sembol	Görünürlük	Anlamı
+	public	Her yerden erişilebilir.
-	private	Sadece sınıfın kendi içinden erişilebilir.
#	protected	Sınıfın kendisi ve alt sınıflar erişebilir.
~	package / internal	Aynı paket (proje) içinden erişilebilir.

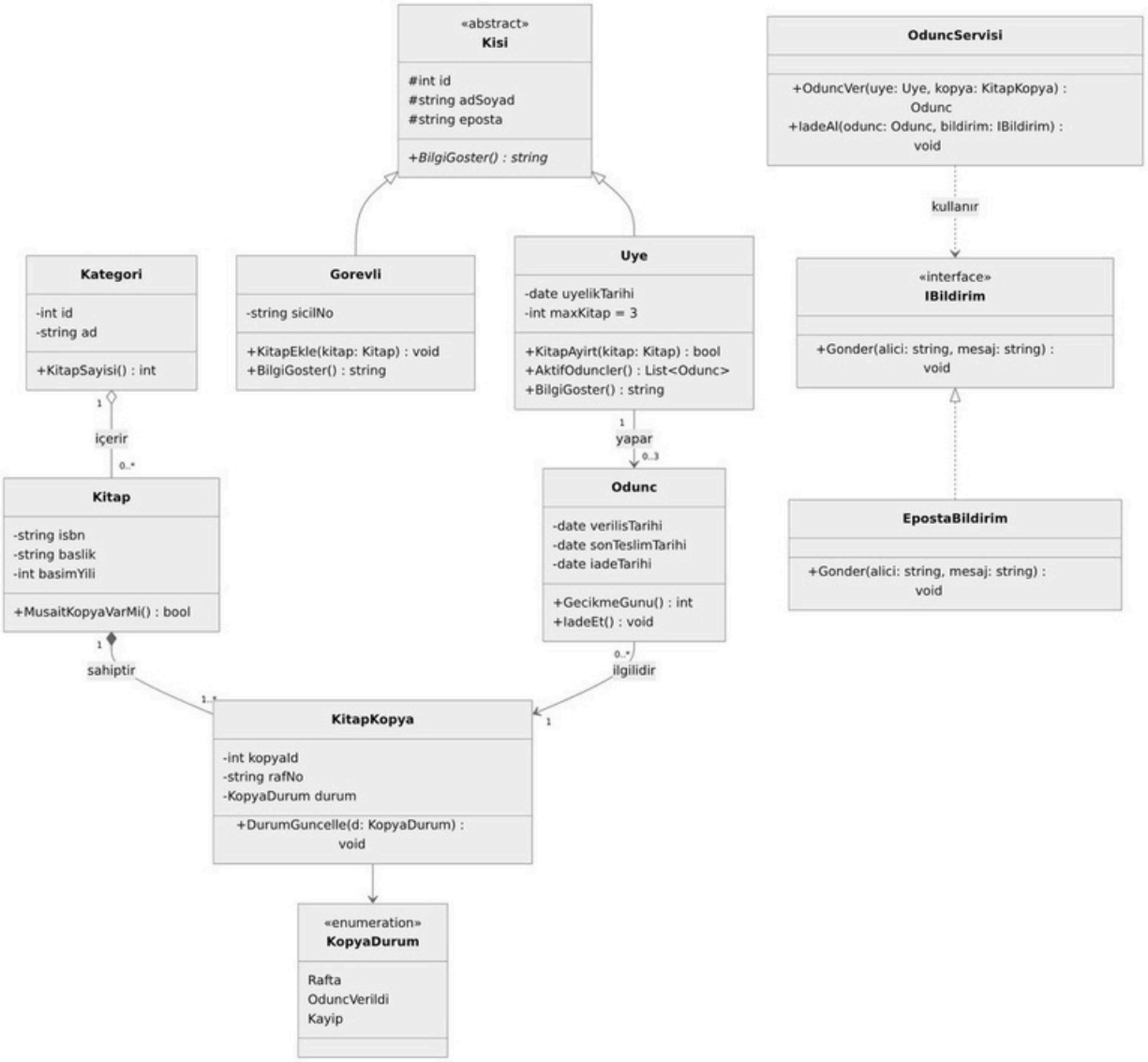
4.2Sınıflar Arası İlişkiler

İlişki	Gösterimi	Anlamı ve Örnek
Association (Birliktelik)	Düz çizgi / oklu çizgi	İki sınıfın birbirini tanıması. Üye, Ödünç kayıtlarını bilir.
Aggregation (Kümelenme)	İçi boş elmas	Zayıf "sahip olma". Parça bütünden bağımsız yaşayabilir. Kategori silinse de kitaplar var olur.
Composition (Bileşim)	İçi dolu elmas	Güçlü "sahip olma". Parça bütün olmadan var olamaz. Kitap silinirse kopyaları da silinir.
Generalization (Kalıtım)	İçi boş üçgen uçlu düz ok	"is-a" ilişkisi. Üye bir Kişidir.
Realization (Gerçekleştirme)	İçi boş üçgen uçlu kesikli ok	Sınıfın bir interface'i uygulaması. EpostaBildirim, IBildirim'i uygular.

İlişki	Gösterimi	Anlamı ve Örnek
Dependency (Bağımlılık)	Kesikli ok	Bir sınıfın diğerini geçici olarak (ör. metot parametresi) kullanması.

Çokluk (Multiplicity): İlişki çizgisinin uçlarına yazılan sayılar, bir nesnenin karşı taraftaki kaç nesneyle ilişkili olduğunu gösterir: 1 (tam bir), 0..1 (sıfır veya bir), 0..* veya * (sıfır veya daha fazla), 1..* (en az bir), 0..3 (en fazla üç).

4.3 Örnek: Kütüphane Yönetim Sistemi Class Diyagramı



Şekil 3. Kütüphane Yönetim Sistemi – Class Diyagramı

4.4 Diyagramın Açıklaması

- **Kalıtım:** Kisi soyut (abstract) bir sınıftır; doğrudan nesnesi oluşturulamaz. Uye ve Gorevli sınıfları Kisi'den türer ve ortak alanları (#id, #adSoyad, #eposta) miras alır. Bu alanlar **protected (#)** tanımlandığı için alt sınıflar erişebilir. Soyut BilgiGoster() metodu her alt sınıfta farklı şekilde yazılır (çok biçimlilik).
- **Composition:** Kitap ile KitapKopya arasında içi dolu elmas vardır. Bir kitabın en az bir kopyası (1..*) bulunur; kitap kütüphaneden silinirse kopyaların da anlamı kalmaz.
- **Aggregation:** Kategori ile Kitap arasında içi boş elmas vardır. Bir kategori silinse bile kitaplar başka bir kategoriye taşınarak varlığını sürdürebilir.

- **Association ve çokluk:** Bir Uye aynı anda **0..3** ödünç kaydına sahip olabilir. Böylece "bir üye en fazla 3 kitap alabilir" iş kuralı diyagramda gösterilmiş olur. Her Odunc kaydı tam olarak bir KitapKopya ile ilişkilidir.
- **Realization:** IBildirim bir arayüzdür; EpostaBildirim sınıfı bu arayüzü uygular (kesikli çizgi + boş üçgen). İleride SMS ile bildirim istenirse sadece yeni bir SmsBildirim sınıfı eklemek yeterli olur.
- **Dependency:** OduncServisi, IadeAl() metodunda parametre olarak bir IBildirim alır ve onu sadece o metotta kullanır. Bu geçici kullanım kesikli okla gösterilen bağımlılıktır.
- **Enumeration:** KopyaDurum bir enum'dur ve bir kopyanın alabileceği sabit değerleri (Rafta, OduncVerildi, Kayip) tanımlar.

Diyagramdan koda dönüşüm örneği (C#)

```
publica bstract class Kisi {

      protected int id;
      protected string adSoyad;
}     protected string eposta;
      public abstract string BilgiGoster();
public class Uye : Kisi
{
                                                    // Generalization

      private DateTime uyelikTarihi;
      private int maxKitap = 3;
}     private List<Odunc> oduncler = new();         // Association (0..3)
public class Kitap
{     public override string BilgiGoster() => $"Üye: {adSoyad}";
      public bool KitapAyirt(Kitap kitap) => oduncler.Count < maxKitap;

}



      private string isbn, baslik;
      private List<KitapKopya> kopyalar = new(); // Composition (1..*)
      public bool MusaitKopyaVarMi() => kopyalar.Any(k => k.Durum == KopyaDurum.Rafta);
```

5. Üç Diyagramın Karşılaştırılması

Kriter	Use Case Diyagramı	ER Diyagramı	Class Diyagramı
Türü	UML – Davranışsal	Veritabanı modeli (UML dışı)	UML – Yapısal
Cevapladığı soru	Sistemi kim kullanır, ne yapar?	Hangi veriler, nasıl saklanır?	Yazılım hangi sınıflardan oluşur?
Bakış açısı	Kullanıcı / müşteri	Veritabanı tasarımcısı	Yazılım geliştirici
Temel elemanlar	Aktör, use case, sistem sınırı	Varlık, nitelik, anahtar, ilişki	Sınıf, özellik, metot, ilişki
Ne zaman çizilir?	Gereksinim analizi	Veritabanı tasarımı	Yazılım tasarımı
Neye dönüşür?	Kullanım senaryoları, test senaryoları	SQL tabloları	C# / Java sınıfları

6. Sonuç

UML diyagramları, bir yazılımı farklı açılardan görmemizi sağlar. Aynı kütüphane sistemi için çizilen üç diyagram aynı sistemi üç farklı soruya göre anlatmaktadır: **Use Case diyagramı** sistemin kullanıcılara ne

sunduğunu, **ER diyagramı** verilerin veritabanında nasıl saklanacağını, **Class diyagramı** ise yazılımın hangi sınıflardan oluşacağını ve bu sınıfların birbirleriyle nasıl çalışacağını göstermektedir.

Bu diyagramların kodlamadan önce hazırlanması, proje ekibinin aynı şeyi anlamasını sağlar ve tasarım hatalarını en ucuz aşamada yakalamaya yardımcı olur. Örneğin “bir üye en fazla 3 kitap alabilir” kuralı use case diyagramında üyelik kontrolü olarak, class diyagramında 0..3 çokluğu olarak ve kodda bir kontrol olarak tutarlı şekilde ifade edilmiştir.

Kaynakça

1. Chen, P. P. (1976). The entity-relationship model – toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36.
2. Fowler, M. (2003). UML distilled: A brief guide to the standard object modeling language (3. baskı). Boston: Addison-Wesley.
3. Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. Erişim adresi: https://www.omg.org/spec/UML/ (Erişim tarihi: 28.10.2026)
4. Sommerville, I. (2016). Software engineering (10. baskı). Boston: Pearson. (Bölüm 5: System Modeling)